Task 1: Problem Definition and Approval

- Define a meaningful real-world software quality problem

Embedded computer vision (CV) systems, like those used currently in autonomous robots, drones, and smart cameras, combine resource-strained hardware (microcontrollers, single-board computers) with computationally intensive vision models (such as YOLOv8, or MediaPipe) to make real-time decisions. In lots of development workflows, quality assurance for these systems focuses almost entirely on functional requirements: does the model detect and classify objects accurately? Non-functional requirements such as latency, frame-rate stability, throughput under load, and resource usage (CPU/memory), are frequently treated as inferior concerns, tested improperly or not at all. This is an important gap. A detection that is accurate but arrives too late, or a system that degrades under load, is a genuine failure in a real time context, not just a missed opportunity.

- Explain the background, motivation, stakeholders, users, and practical relevance of the selected problem

- Background

Real time embedded CV is common in robotics, autonomous navigation, robots, and drones, where timing and resource constraints are as critical as detection accuracy. A model that performs well during the benchmark can fail in deployment if it can't keep a consistent frame rate on constrained hardware, or if performance degrades unpredictably as the system load grows. This problem is motivated by direct exposure to this context, where the disconnect between "the model works" and "the system meets its real time requirements" is a recurring, practical concern.

- Stakeholders and Users

  Developers/Engineers – Build embedded CV pipelines, who need confidence that functionality doesn't come at the cost of performance in real time

  QA/Test engineers – Need a defined, repeatable way to test NFRs alongside FRs

  End users – Affected if the system misses detections due to latency or degrades under bigger loads

  Safety/compliance reviewers – In contexts where late or missed detections have real consequences

- Practical Relevance

This is a widely common problem across all forms of robotics. Any domain where CV runs on embedded hardware under real time constraints will eventually encounter issues. Addressing it produces a testing approach that is usable well beyond a single interaction.

• Identify the key software quality issues involved in the problem

- Key Software Quality Issues

  Absence of clearly defined testable non-functional requirements for real-time CV systems

  Lack of methods to measure and validate latency, framerate, and resource usage.

  Difficulty differentiating between “the model is accurate” from “the system meets the real time requirements”

  Risk of performance degradation under load or resource allocation.

  Absence of defect management processes specifically for NFR

• Explain why the problem is suitable for a Software Quality Assurance Project

The problem centres on requirements quality (defining testable NFRs), test strategy design (functional vs non-functional approaches), defect management (tracking NFR violations differently from functional bugs), and quality metrics (latency thresholds, degradation curves) which all are core SQA requirements. It also supports a requirements traceability matrix, since both functional and non-functional requirements can be trace to measurable test cases.

• Confirm that the project is achievable within one semester

The project is a software-simulated prototype rather than physical embedded hardware, removing cost and hardware-access constraints. By using pre-recorded video/image datasets, an already existing lightweight CV mode, and simulated resource constraints, we can achieve the technical build within a few weeks, leaving the rest of the semester for QA artifacts (requirements analysis, test strategy, test cases, traceability matrix, defect management and metrics) which is where the assessment is mostly centred around.

Task 2: Requirements and Quality Analysis

- ## Initial Functional Requirements

ID	Requirement
FR-01	The system shall load and run a pre-tainted object detection model (e.g., YOLOv8n or MediaPipe) on input video/image data.
FR-02	The system shall detect and classify objects present in each processed frame.
FR-03	The system shall output bounding boxes, class labels, and confidence scores for each detected object.
FR-04	The system shall process video input from a pre-recorded file or image sequence, simulating a live feed.
FR-05	The system shall log each frame's detection results and associated timestamp for later analysis.
FR-06	The system shall allow configuration of a simulated resource constraint (e.g., limited CPU threads, memory cap) before a test run.
FR-07	The system shall generate a summary report of detection accuracy and performance metrics after a test run completes.
FR-08	The system shall allow a user to select between at least two load conditions (e.g., baseline vs high-load) for a given test run

- ## Initial non-functional requirements

ID	Requirement	Quality attribute
NFR-01	The system should process frames quickly.	Performance
NFR-02	The system should maintain a stable frame rate.	Performance and Reliability
NFR-03	The system should not use to much memory.	Resource Efficiency
NFR-04	The system should keep working under high load.	Reliability
NFR-05	The system should be easy to configure for different test scenarios.	Usability
NFR-06	The system should produce consistent results across repeated runs.	Reliability
NFR-07	The system should be portable across different simulated hardware profiles.	Portability

- ## Requirement Analysis

Each requirement was assessed against six quality criteria: clarity, completeness, consistency, correctness, feasibility, and testability.

- ## Clarity

The requirements labelled FR-01 to FR-08 are highly clear and unambiguous because they use strong action verbs such as explicitly stated operational phrases, "shall load". A contrast to these statements, the subsequent NFRs between 01 and 04 are mainly based on Vague qualifiers (example but not all were used) such as "quickly", "too much", "stable" and "keep working". As a result, this type of vague language unwittingly renders these requirements particularly vulnerable to the potentially various subjective interpretations while they also suffer from an absence of any well-articulated numerical parameters or objective performance standards.

- ## Completeness

While the Functional Requirements defined so far satisfy most of requirements corresponding to critical logic and common flow in detection pipeline, it is still severely limited by its incompleteness especially when it comes to handling errors. There is a complete lack of any formal specification for required system behaviour and expected automated remedies in case some specified frame were to fail an assertion that it can be processed correctly, the core underlying model file would turn out to just not exist at all or else incoming stream of input data could actually be found to have been corrupted.

- ## Consistency

No requirements were found to be in conflict. But FR-06 (configurable resource constraint) and NFR-03("should not use too much memory") are conceptually overlapping but explicitly not linked.

- ## Correctness

The requirements accurately reflect the problem that was defined in Task 1(The FR/NFR imbalance in embedded CV). No requirement was found that misinterpreted the intended system behaviour

- ## Feasibility

The FRs are all within the bounds of what can be verified on a software simulation prototype using recorded datasets (in line with agreed upon boundaries in Task 1). NFR-07 (portability across simulated hardware profiles) is only possible if ""hardware profile" means software-level constraints, like CPU/memory throttling instead of true cross-device deployment.

- ## Testability

This is the weakest point. This is because none of NFR-01 to NFR-05 have a defined numerical value and cannot be tested with absolute pass/fail. This is an expression of

the fundamental issue identified in Task 1: NFCs exist, but do not appear in a form suitable for QA validation.

- ## Rewritten Requirements

ID	Rewritten Requirements (Testable)	Quality attribute
NFR-01	The system should process each frame under 100ms on the hardware profile	Performance
NFR-02	The system should sustain a minimum of 15 FPS for at least 95% of its frames	Performance and Reliability
NFR-03	The system should not exceed 512MB of memory usage during a test run	Resource Efficiency
NFR-04	The system should complete detection on 100% of its input frames without crashing when under a high stress load.	Reliability
NFR-05	A user should be able to configure and do a test run (selecting resource constraint and load condition) using no more than 3 configuration steps, without having to edit the source code	Usability
NFR-06	Given similar input and configurations, the system should produce detection outputs (class labels, bounding boxes) that match roughly a 2% variance across 5 repeated runs	Reliability
NFR-07	The system should complete a full test run without fail, under at least 2 distinct simulated hardware profiles (normal and constrained CPU/Memory)	Portability
NFR-08	The system should properly handle all error conditions (missing model, corrupted frame, invalid input path) without crashing in 100% of test cases.	Reliability
NFR-09	The system shall complete model loading and first-frame inference (cold-start) in under 500ms	Performance

ID	Requirement
FR-06	The system shall allow configuration of a simulated resource constraint (e.g., limited CPU threads, memory cap defined NFR-03) before a test run.
FR-09	If the model file can't be located or can't load, the system should stop the test run and log an error message rather than failing silently
FR-10	If an input frame is corrupted or unreadable, the system should skip that frame, log the failure, and continue processing other frames
FR-11	If input video/image data is missing or the file path is invalid, the system should reject the test run before running and notify the user

- FR Acceptance criteria for key features

Feature: Object Detection Pipeline (FR-01, FR-02, FR-03, FR-04)

- Given a valid pre-recorded video/image, the system loads the specified detection model and processes every frame without human intervention.
- Each processed frame creates bounding boxes, class labels, and confidence scores for all detected objects.
- Detection results are visually and numerically verifiable against a manually labelled sample of frames (spot-check accuracy).

Feature: Detection Logging (FR-05)

- Every processed frame creates an entry log containing a timestamp and detection results.
- Log entries are readable and retrievable after a test run is done (as a CSV/JSON file).

Feature: Resource Constraint Configuration (FR-06)

- A user can select a resource constraint profile (baseline vs constrained CPU/Memory) before starting a test run, without having to edit the source code.
- The selected constraint profile is properly applied and reflected in the systems runtime configuration for that test run.

Feature: Load Condition Selection (FR-08)

- A user can choose between at least two load conditions (baseline vs high load) before a test run.
- The selected load condition is properly applied to the test run's execution.

Feature: Performance Summary Report (FR-07)

- After a test run completes, a summary report is made containing detection results and logged performance data.
- The report is generated automatically without any manual intervention.

Feature: Error Handling

- If the model file is missing or invalid, the run stops and an error message is logged.
- If a frame is corrupted, it's skipped and logged, and processing continues onto the other frames.
- If the input path is invalid, the run is halted before its executed, with a message to the user.

- ## NFR Acceptance criteria for key features

Performance (NFR-01, NFR-02)

At least 95% of processed frames are completed in under 100ms, and frame rate does not drop below 15 FPS during a baseline run.

Resource Efficiency (NFR-03)

Peak Memory usage does not exceed 512MB during a test run.

Reliability – Load (NFR-04)

Under the high load configuration, 100% of frames are processed without a crash.

Configurability (NFR-05)

A user can configure and launch a test run in no more than 3 steps, without having to edit the source code.

Consistency (NFR-06)

Repeated runs with identical input and configuration produce results within 2% variance.

Portability (NFR-07)

The system completes a full test run under at least two distinct simulated hardware profiles.

Reliability – Fault Handling (NFR-08)

All defined error conditions are handled without crashing in 100% of test cases.

- ## Software Quality Attributes

Performance Efficiency

The system's ability to process frames within time constraints and use resources properly. Directly covered by NFR-01 (frame processing time), NFR-02 (frame rate stability), and NFR-03 (memory usage).

Reliability

The system's ability to maintain function under stated conditions, including fault tolerance and recoverability. Covered by NFR-04 (load handling), NFR-06 (consistency across runs), and NFR-08 (fault/error handling), plus the error-handling FRs (FR-09, FR-10, FR-11).

Usability

Ease of configuration and use by a tester/operator without needing to modify code. Covered by NFR-05 (configuration simplicity) and FR-06/FR-08 (resource constraint and load condition selection).

Portability

The system's ability to run correctly across different simulated hardware/resource profiles. Covered by NFR-07.

Maintainability

Not explicitly covered by a numbered NFR, but relevant given the project's iterative development. The codebase should remain modular enough that resource constraint profiles, load conditions, and detection models can be swapped or extended without a lot of work.

Not applicable

Security, Accessibility, and Compatibility since the prototype is a local, simulated testing tool without user facing security surfaces, accessibility-relevant UI, or cross compatibility requirements.

Task 3: Proposed Solution and Initial Prototype

- ## How the Solution Addresses the Problem

The proposed solution targets the gap that we identified in Task 1. Embedded CV systems are usually tested for functional accuracy but not for real-time performance behaviour. The prototype shows a detection pipeline fitted with performance logging and configurable resource/load conditions, allowing both functional correctness and non-functional threshold (latency, frame rate, memory, reliability under load) to be tested against a measurable criteria rather than an informal observation. This turns an overlooked NFR gap into something testable which is the core of the project.

- ## Prototype Overview

The initial prototype consists of four core modules, aligned with the FRs from Task 2:

1. Detection Pipeline – loads a pre-trained lightweight model (e.g. MediaPipe or YOLOv8n) and processes pre-loaded video/images frame by frame, producing bounding boxes, class labels, and confidence scores (FR-01, FR-02, FR-03, FR-04).

2. Resource Constraint Simulator – applies configurable, software level constraints (e.g. CPU thread limiting, memory capping) to emulate embedded hardware conditions without needing physical hardware (FR-06)).
3. Metrics Logging Module – records per-frame timestamps, detection results, latency, and resource usage throughout a test run (FR-05).
4. Summary Reporting – After a run completes, compiles logged data into a report showing detection results and performance metrics, including pass/fail status against NFR thresholds (FR-07)

Error handling (FR-09, FR-10, FR-11) is implemented across these modules. For example, the detection pipeline validates the model file on load, and the logging module records skipped/corrupted frames rather than stopping the run entirely.

- ### Interface Justification: GUI over CLI/Dashboard

Since the solution involves configuration, workflow execution (running a test), and reporting/dashboard style output, a GUI is more appropriate against the brief. The prototype will include a lightweight dashboard interface (probably a simple web front end) that allows users to:

- Select a resource constraint profile and load condition before starting a new test run (satisfying NFR-05's configurability requirement)
- Starting a test run without needing to edit code or use a terminal
- View live or post-run detection results (bounding boxes/labels on sample frames)
- View a performance summary showing latency, frame rate, memory usage, and pass/fail status against NFR thresholds.

This keeps the interface simple and focused on the QA workflow (configure → run → review results) whilst satisfying the GUI requirement in a way that's useful rather than just for the sake of ticking a box.

Task 4: AI-Assisted Development Using GitHub Copilot

- ### How AI-Assisted Development was used

GitHub Copilot and Claude (via chat-based pair programming) was used throughout prototype development to help with coding and help debug new unfamiliar library APIs. AI assistance supported nearly every aspect of the prototype: the detection pipeline, resource constraint simulator, metrics logging, summary reporting, load condition handling, error handling, and the MAUI GUI.

• Parts of the prototype supported by AI suggestions

- Initial scaffolding of OnnxYoloDetector, DetectionResult, IDetector, and the overall structure.
- Debugging assistance where the YoloDotNet library's actual API (v4.2.0) didn't match the documentation, Copilots direct access to installed package IntelliSense properly identified CpuExecutionProvider and its constructor, fixing lots of build errors that the generic documentation search couldn't
- The ResourceConstraintSimulator, MetricsLogger, SummaryReportGenerator, and LoadCondition processing logic
- MSTest test cases verifying each module (detection, pipeline, resource constraint, load conditions, error handling)
- The MAUI GUI (MainPage.xaml, MainPage.xaml.cs

• Example prompts and AI-assisted outputs

- Copilot is recommending I use "new CpuExecutionProvider(). Copilot properly identified the actual implementation class since Claude guessed earlier that YoloExecutionProvider was incorrect, based on live access to the installed NuGet package rather than outdated documentation.
- Decompiled source inspection (via VS's Go to Definition) was used to verify actual class/constructor signatures (YoloOptions, CpuExecutionProvider, ObjectDetection) rather than relying on assumed API shapes
- AI assisted diagnosis of a concurrency bug: initial NFR-01 latency measurements were inconsistent (890ms to 3001ms average) across test runs. By isolating the test, reproducing the bug and then comparing traced this back to MSTest's parallel test execution allowing ResourceConstraintSimulators process wide ProcessAffinity change to leak into unrelated tests running at the same time was fixed via [assembly: DoNotParallelize]

• Limitations, errors, and risks found in AI-Assisted code

- Initial Claude generated code (YoloOptions and SixLabors.ImageShape) was based on outdated assumptions for YoloDotNet library's API and did not match the actual installed version (v4.2.0 uses SkiaSharp, not ImageSharp, and a different YoloOptions structure than assumed). It needed multiple rounds of correction using the actual decompiled source
- AI suggested code cause a cross-thread concurrency bug (CPU affinity leaking across parallel tests) that wasn't caught until the inconsistent, hard to reproduce test results instigated human investigation. This serves as a reminder that AI-suggested code should be tested under real conditions and not just in isolation

- Placeholder NFR thresholds were initially set before real data was found, and needed revision once actual data was available. AI suggestions for thresholds without background evidence should be treated with a grain of salt, not set in stone.

- ### Reflection on AI-assisted coding

AI assistance really pushed this project fast, being done in a day, with help in unfamiliar third-party library APIs (YoloDotNet) where documentation was sparse or outdated. By directly inspecting decompiled source via Copilot/IntelliSense proved more dependable that general purpose AI suggestions based on training data alone. The concurrency bug showed a key limitation: AI-generated code can pass isolated tests but can still carry hidden, environment dependent bugs that only show under real conditions. This highlights the importance of human verification, by running the full test runs, double checking inconsistent results, and looking further into root causes rather than taking AI suggestions at face value.

Task 5: Initial Test Strategy and Test Planning

Scope of Testing:

- The testing scope includes: The 4 fundamental modules from Task 3, being the detection pipeline, the resource constraint simulator, the metrics logging and the summary reporting. Also in scope is the error handling across them, represented by FR-09 through FR-11. Functional correctness is in scope as well as non-functional characteristics such as latency, frame rate, memory utilization and reliability under pressure.
- Out of scope: The embedded hardware devices that implement embedded the software, security and accessibility testing, and cross platform support apart from MAUI Windows.

Test levels and types:

- Integration Testing: the main level of testing performed so far. Tests are executed on the real OnnxYoloDetector, real model of yolov8n.onnx, and sample images. This is used to verify the interaction between modules.
- Unit Testing: initially designed as a refinement. DetectionPipelineRunner requires the IDetector interface and not a concrete detector, and this permits using a mock detector, thus isolating pipeline, logging, and reporting logic from the real model when the test coverage is sufficient.
- System-level testing: manual verification via the EmbeddedCV.MAUI GUI, from configure-run-review, of the entire end-to-end workflow.

- Regression testing: running the suite of MSTests again after any modification; to ensure that problems such as the cross-test interface which occurred during Task 4 do not arise.
- Non-functional testing: threshold and comparison-based checks which are either asserted directly in the test methods or derived by SummaryReportGenerator.

Functional testing approach:

- Functional tests pass real simple images to the detect function and make assertions on the detection output, number of frames, and the contents of the JSON log, respectively. Error-handling tests deliberately pass malformed inputs (no model or image path provided) and check for the appropriate exception type being thrown. Input validation prevents the batch from ever starting at all if a mandatory file is missing (FR-11); per-frame failures are caught and logged as skipped rather than stopping the run to continue (FR-10).

Non-functional testing approach:

- Latency and cold-start timing (NFR-01, NFR-09) are measured on each frame, tested against the given values in SummaryReportGenerator. Constrained behaviour is also tested with a check and is designed so that: a constrained-profile run exhibits high latency in comparison to the non-constrained run. Reliability under load (NFR-04) checks that every frame completed and was not dropped when execution was either sequential or concurrent.
- Memory usage (NFR-03) has a value assigned but has yet to be coded into the logic for measuring that value. Also, NFR-08 currently merely checks that an unhandled exception did not bring the run to its end as opposed to the full check required. Both these are designated future tasks rather than being errors from this run.

Entry Criteria:

- Accepted FRs/NFRs exist in Relevant FRs/NFRs (Task 2).
- Existing model file and sample images are stored as expected.
- EmbeddedCV.Tests build against EmbeddedCV.Core successfully.

Exit Criteria:

- Execution of all planned test cases in the current cycle and recording their results.
- Functional test cases pass or when failures are logged as defect and severity assigned.

- Non-functional test cases have an associated measured value against the threshold limit where there is a threshold.
- No high-severity defects remain open
- Results are captured and visible to everyone in the requirements traceability matrix; no gaps in coverage are assumed, such as NFR-02 through NFR-07 that have not yet been tested

Test environment, tools and dependencies:

- IDE: Visual Studio, used for development and running/debugging the MSTest suite via Test Explorer.
- Language/runtime: C# on .NET 9.
- GUI framework: .NET MAUI, Windows target.
- Detection model: YOLOv8n (`yolov8n.onnx`) via YoloDotNet (v4.2.0) on ONNX Runtime, `CpuExecutionProvider`.
- Test framework: MSTest, in `EmbeddedCV.Tests`, referencing `EmbeddedCV.Core` directly.
- Test data: model file under `Assets/models/`; sample images (`bus.jpg`, `zidane.jpg`) under `EmbeddedCV.QA/EmbeddedCV.Tests/SampleData/` for the test project, and `Assets/sample-data/` for the MAUI project.
- Version control: GitHub.

Task 6: Initial Test Cases and Requirements Traceability

- ## Test Cases

ID	Test Case	Requirements	Type
TC-01	Detector returns valid detections on known sample image (bounding box, label, confidence)	FR-01, FR-02, FR-03, FR-04	Functional
TC-02	Full pipeline processes multiple frames and gives valid JSON output (Detect → time → log)	FR-04, FR-05	Functional/Integration
TC-03	Constrained resource profile gives higher latency than baseline profile	FR-06, NFR-01	Non-functional
TC-04	Summary report evaluates NFR-01 (latency) as pass/fail against threshold	FR-07, NFR-01	Non-functional
TC-05	Cold-start latency measured and evaluated separately from steady state latency (NFR-09)	FR-07, NFR-09	Non-functional
TC-06	Batch processing under Baseline and HighLoad conditions both complete all frames without error	FR-08	Functional
TC-07	Detector constructor throws an error when model file is missing	FR-09	Negative/Error handling
TC-08	Pipeline stops a run before running when input file path is invalid	FR-11	Negative/Error handling
TC-09	System handles error conditions across runs without crashing (NFR-08)	FR-09, FR-10, FR-11, NFR-088	Reliability

- ## Requirements Traceability Matrix

Requirement	Description	Test Case
FR-01	Load and run detection model	TC-01
FR-02	Detect/classify objects per frame	TC-01
FR-03	Output bounding boxes, labels, confidence	TC-01
FR-04	Process video/image input	TC-01, TC-02
FR-05	Log per frame timestamp + results	TC-02
FR-06	Configure resource constraint	TC-03
FR-07	Generate summary report with metrics	TC-04, TC-05
FR-08	Select between load conditions	TC-06
FR-09	Stop and log error on missing/invalid model	TC-07, TC-09
FR-10	Skip corrupted frame, continue processing	TC-09
FR-11	Reject run before running on invalid input path	TC-08, TC-09
NFR-01	Frame processing under 100ms (steady state)	TC-03, TC-04
NFR-08	Manage errors without crashing	TC-09
NFR-09	Cold-start latency under threshold	TC-05

NFR-02 through to NFR-07 are untested, we will evaluate them properly in the next phase since they need more structure that has not been written yet.

- ## Why traceability matters

A requirements traceability matrix supports quality assurance by making test coverage verifiable rather than assumed. It shows which requirement each test case validates, and which requirements still haven't been covered, rather than letting the gaps be. It also supports change management: when a requirement changes like with NFR-09's cold start threshold, the requirements traceability matrix shows which test cases need re-running or updates instead of needing review of all of them. This becomes essential

as the project grows, stopping changes from silently failing unrelated coverage. Overall, the RTM gives a clear trail from Task 1's problem, through Task 2's requirements, to Task 6's evidence that the requirements are being validated.

Task 7: Project Progress, Risks, and Next Steps

- ## Work summary

The group came to agree on a real-world software quality problem centred on non-functional requirement testing gaps in embedded computer vision systems. Initial functional and non-functional requirements were found and analysed for clarity, completeness, consistency, correctness, feasibility, and testability, with many NFRs rewritten to include thresholds and acceptance criteria defined for key features (Task 2). An initial prototype was made in C#/.NET9 using .NET MAUI (Windows), consisting of YOLOv8n (via YoloDotNet/ONNX Runtime), a metrics logging system, a resource constraint simulator (Baseline vs Constrained profiles), a summary report generator evaluating results against NFR thresholds, error handling, and a GUI dashboard allowing users to configure and run test scenarios (Task 3). GitHub Copilot was used in development to speed up coding and debug third party library integration issues (Task 4). Nine MSTest cases were created covering functional and non-functional requirements, and an initial requirements traceability matrix was made linking requirements to assess evidence (Task 6). Assessing the prototype gave us real data: baseline conditions met the NFR-01 latency threshold (79ms avg against <100ms), whilst constrained and high load conditions caused NFR failures (up to ~2150ms avg latency), showing the real-world quality problem the project is investigating.

- ## Current limitations, risks, and challenges

Limited NFR Coverage: Only NFR-01, NFR-08, and NFR-09 are currently assessed. NFR-02 through NFR-07 haven't been assessed yet.

Provisional thresholds: Some NFR thresholds (NFR-05, NFR-06) were given placeholders and stay as estimates.

Small sample dataset: Testing so far has relied on only two sample images. This is enough to show functionality but not enough to prove real time processing.

Test isolation risk: A concurrency bug was found where parallel test execution allowed process-level state from one test to leak into another, this has been fixed but shows a broader risk of test interference as tests grow.

Simulated rather than physical constraints: Constraints are simulated rather than tested on embedded hardware, which won't reflect real world embedded performance.

Model licensing: The YOLOv8 model used is subject to Ultralytics commercial licensing terms.

- Risk management

Risk	Likelihood	Impact	Mitigation
NFR-02 to NFR-07 remain untested	Medium	High. Leaves the traceability matrix with visible coverage gaps against the project's stated goal	Prioritise NFR-02 (frame rate) and NFR-07 (portability) first, since they reuse existing pipeline/simulator infrastructure; schedule remaining NFRs across the final sprint rather than left for the end
Provisional thresholds (NFR-05, NFR-06) are accepted without real measurement	Medium	Medium. Undermines the testability claims made in task 2 if left unvalidated	Collect real data for configuration step counts (NFR-05) and variance of repeated-runs (NFR-06) before finalising the thresholds
Smal sample dataset (two images) limits confidence in detection accuracy claims	Medium	Medium. Spot-check accuracy checks are not statistically meaningful on this small set	Expand the sample set with a wider range of frames (varied lighting, object density, occlusion) before the final testing
Test isolation issues recur as the suite grows	Low-Medium	High. Already caused a hard to reproduce concurrency bug	Resolve the conflicts between MSTestSettings.cs (Parallelize) and AssemblyInfo.cs (DoNotParallelize) before adding anymore tests; keep DoNotParallelize unless a valid reason to parallelise is confirmed
NFR-03 (memory) has a defined threshold but no implemented measurement	Medium	High. A defined but unmeasured NFR is a direct gap against the testability argument made in Task 2	Implement memory measurement (e.g. Process.Workingset64 sampling) in SummarReportGenerator before claiming NFR-03 as tested

NFR-08 check currently only confirms the run didn't throw unhandled, not that each error condition was correctly handled	Low	Medium. Risk of reporting a false pass on reliability	Extend the NFR-08 check to verify each error path (missing model, corrupted frame, invalid path) rather than marking success from completion alone

Plan for Final Project

- Develop and run tests for NFR-02 - NFR-07 and fill in the coverage gap identified in the limitations for Task 6 and Task 7.
- Measure the memory usage in SummaryReportGenerator so that there is a tangible check implemented in NFR-03, rather than a stipulated but unutilized threshold.
- Ensure NFR-08 does not rely on merely not receiving an error; implement tests that check specifically for each stated error condition (non-existent model; corrupted frame; non-existent path).
- Resolve MSTestSettings.cs / AssemblyInfo.cs parallelization conflict and confirm test isolation is still reliable for the entire test suite.
- Expand sample image set from 2 to cover more scenarios for NFR-06 and detection accuracy;
- Re-evaluate/validate provisional thresholds proposed for NFR-05 and NFR-06 against measured data.
- Test up to the unit level using the IDetector abstraction, to abstract away pipeline, reporting, and logging logic, not involving real model:
- Update traceability matrix: include coverage; reconcile vs. Total number of tests
- Formalize an NFR-specific defect reporting procedure clearly distinguishes NFR violations to address the problem stated in Task 1.
- Assemble all the various sections into final report, incorporating any remaining fixes from this revision;

Team AI reflection

AI (GitHub Copilot and Claude) was used throughout the project, from prototype development to test deign, and report drafting. It was reliable when grounded on something concrete, verifiable, and untrustworthy when depending solely on memory: in the case of Copilot, inspection of the actual installed package for YoloDotNet

resolved some real build errors, whereas the initial scaffolding from Claude was based on an outdated API and required several levels of correction.

The most obvious flaw of the concurrency bug in Task 4 was that all the AI-proposed code passed merely in isolation but failed once subjected to true parallel execution, and it was only actual investigation that eventually led to the source of the problem. This firmly established the fact that all AI-Assisted code must be tested under actual circumstances, rather than examined simply by inspection.

Similar was done to the thresholds; the placeholder NFR values were adjusted after data collection instead of being left to AI.

AI was useful in progressing on unfamiliar APIs and first drafts, but each AI-proposed fact or line of code was a piece to be confirmed, not an answer.